

SMARTRETAIL

Intelligent Electronics POS & Installment Management Platform

Complete Project Proposal

A transaction-safe, workflow-orchestrated retail platform with predictive inventory intelligence, financial reporting, and production-oriented engineering.

Project Type	Software Engineering / Intelligent Systems
Architecture	Modular Monolith + Background Workers
Primary Stack	React, NestJS, PostgreSQL, Redis, Temporal, Python
Deployment	Docker + GitHub Actions; Kubernetes optional future extension

Prepared as a refined version of the Electronics POS Marketplace Management System proposal. Delivered by a two-member team.

1. Executive Summary

SmartRetail is a scalable electronics retail management platform designed for businesses that handle full-payment and installment-based purchases. It combines POS operations, customer management, serialized inventory, payments, installment schedules, refunds, warranty tracking, invoices, downloadable reports, durable workflow automation, predictive analytics, CI/CD, and structured application logging, with a full metrics/dashboard observability stack as an optional extension.

The project is intentionally designed so that advanced technologies solve real engineering problems. PostgreSQL remains the transactional source of truth; Temporal manages long-running installment workflows; Redis supports caching and short-lived state; Python-based analytics and ML provide decision support; and Docker/GitHub Actions provide reproducible deployment and automation. Kubernetes, microservices, Kafka, an LLM, and a lakehouse are not required for the initial system.

2. Problem Statement

Electronics retailers must keep sales, payments, stock, customer balances, installment schedules, warranties, refunds, and reports consistent. Installment sales are especially difficult because a single purchase creates a business process that can continue for months. Manual reminders and spreadsheet-based tracking can lead to missed payments, inconsistent balances, poor auditability, and unreliable reporting.

Traditional POS systems also tend to report only current stock. They do not explain why stock changed, predict future demand, estimate stock-out risk, recommend replenishment quantities, or highlight unusual sales and inventory activity. SmartRetail addresses these gaps while preserving strong transactional correctness.

3. Project Objectives

1. Build a reliable POS core for sales, customers, products, inventory, payments, and installment plans.
2. Guarantee transaction consistency across checkout, payments, inventory deduction, refunds, and reversals.
3. Automate installment reminders, retries, due dates, and overdue escalation using durable workflows.
4. Generate downloadable invoices, installment receipts, statements, and management reports in PDF, CSV, and XLSX formats.
5. Track serialized electronics units, IMEI/serial numbers, warranty periods, returns, and warranty claims.
6. Provide demand forecasting, stock-out prediction, reorder recommendations, installment-risk scoring, and anomaly detection.
7. Implement RBAC, audit logging, idempotency, concurrency control, and secure handling of customer and financial data.
8. Provide automated testing, CI/CD, and structured application logging as core requirements, with a full production-style observability stack (metrics and dashboards) as an optional extension.
9. Keep the architecture horizontally scalable without making Kubernetes or microservices mandatory.

4. Users and Roles

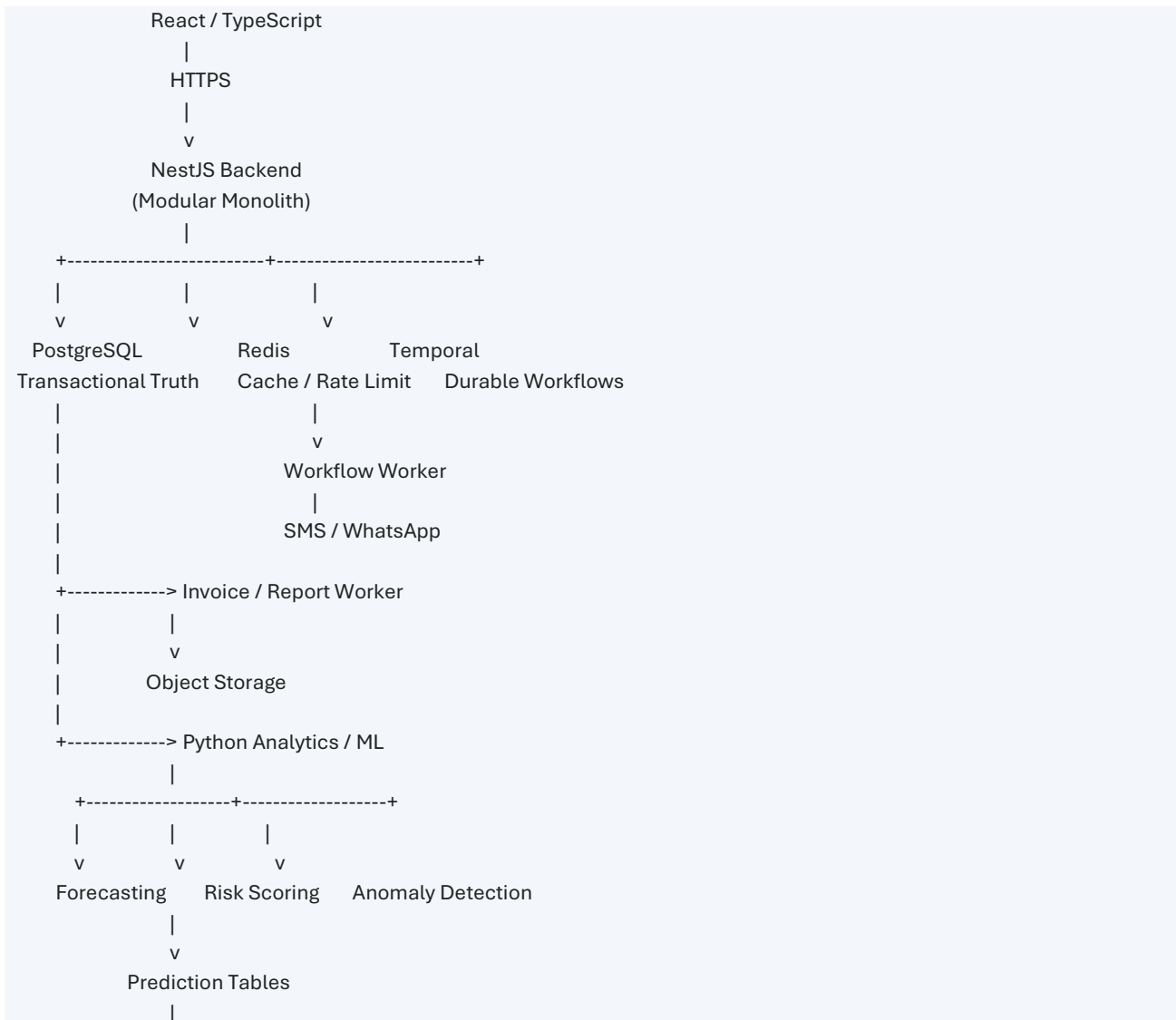
4.1 Administrator

- Manage workers, roles, products, categories, pricing, interest rates, suppliers, and configuration.
- Review sales, payments, refunds, stock adjustments, installment accounts, and audit records.
- Access dashboards, predictive insights, reorder recommendations, and risk alerts.
- Generate and download operational, financial, inventory, and installment reports.

4.2 Worker / Cashier

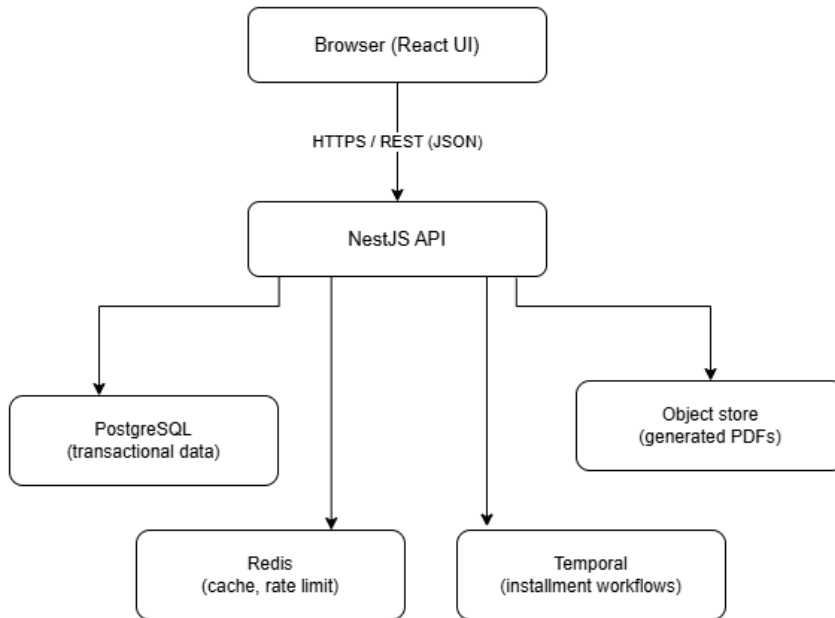
- Search or scan products, register customers, create sales, and receive payments.
- Create installment purchases and record installment payments.
- Print or download receipts and invoices.
- Search serialized products, warranty status, and permitted transaction history.
- Process authorized returns or refund requests.

5. Proposed System Architecture



v
Admin Dashboard

The backend begins as a modular monolith because the business domains need clear boundaries but do not justify independent microservice deployment. Computationally heavy or asynchronous components, such as report generation and ML jobs, run as workers and can be scaled independently later.



6. Core Backend Modules

```

backend/src/
|-- auth/
|-- users/
|-- customers/
|-- catalog/
|-- inventory/
|-- serialized-units/
|-- suppliers/
|-- sales/
|-- payments/
|-- installments/
|-- invoices/
|-- refunds/
|-- warranties/
|-- workflows/
|-- notifications/
|-- reports/
|-- analytics/
|-- intelligence/
|-- audit/
`-- common/
  
```

The repository is a monorepo with top-level frontend/ and backend/ folders (named literally, not apps/web and apps/api), a separate workers/temporal-worker/ process for the installment workflow, and an optional ml-service/ (Python) for the analytics work in Section 15 — each runs as its own process since both need to operate independently of the request/response API.

7. Transaction-Safe POS Processing

Checkout is treated as a financial transaction rather than a sequence of unrelated CRUD operations. Critical database changes must either succeed together or fail together.

```

Checkout
|
Validate customer, products, prices and stock
|
BEGIN DATABASE TRANSACTION
|
Create Sale + Sale Items
|
Record Payment
|
Create Stock Movements
|
Update Inventory
|
Create Audit Entry
|
COMMIT
|
Generate Invoice / Receipt

Any critical failure before COMMIT -> ROLLBACK

```

7.1 Reliability Controls

- Database transactions and suitable isolation levels.
- Optimistic or pessimistic locking for contested inventory.
- Idempotency keys for payment and checkout endpoints to prevent duplicate processing.
- Retry-safe APIs and unique constraints for critical financial identifiers.
- No destructive deletion of completed financial records; use reversals and refunds instead.

8. Inventory and Serialized Electronics

Inventory will be ledger-based. Every stock change creates a StockMovement record instead of silently changing a quantity. This makes inventory explainable and auditable.

```

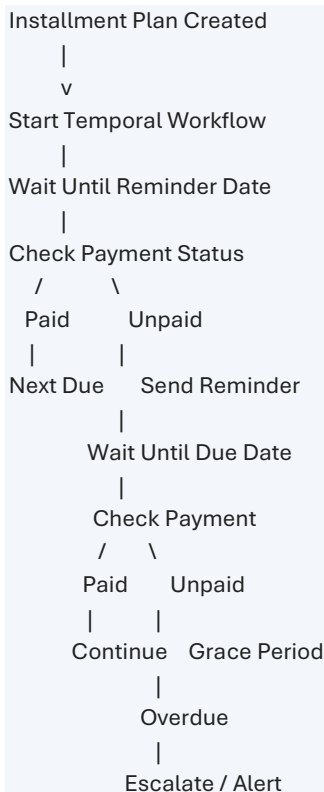
PURCHASE    +20
SALE        -2
RETURN       +1
DAMAGED     -1
ADJUSTMENT  -2
TRANSFER    +/-N

```

Because this is an electronics-specific system, selected products can also be tracked as individual serialized units using serial numbers, IMEI values, warranty periods, and sale status. This supports warranty lookup, returns, product traceability, and more realistic electronics retail workflows. Per the finalized data model, each serialized unit links to the exact sale item that sold it, not just to the product in general, so a warranty lookup or return traces back to the precise transaction; a database constraint prevents the same serialized unit from being sold twice.

9. Installment Management and Temporal Workflows

Each installment plan stores its financial schedule in PostgreSQL. Temporal is used only for the long-running workflow lifecycle: timers, reminders, retries, due-date checks, and overdue escalation. Financial updates remain inside transactional backend logic. Per the finalized design, one workflow runs per installment (not per plan), using that installment's own schedule ID as the workflow ID; Temporal refuses to run two workflows with the same ID, which gives duplicate-reminder prevention without extra code. Payment detection is polling-based (the workflow re-reads the database at each scheduled check) rather than event-driven, since the simpler design already satisfies every requirement here. The workflow itself runs in a separate worker process from the request/response API, since it must keep running independently of it.



9.1 Edge Cases

- Partial payment, early payment, late payment, missed payment, and overpayment.
- Payment reversal and installment-plan cancellation.
- Notification provider failure and retry.
- Application or worker restart while a workflow is waiting.
- Duplicate payment requests and repeated workflow signals.

Overpayment is handled by the payment model itself rather than as a special case: Payments is a single table (a type column distinguishes a full payment, deposit, or installment payment), and a separate PaymentAllocations table maps one payment to one or more installment schedule rows with an amount applied to each. This is what lets a single payment settle one installment in full and prepay part of the next, without a nullable schedule reference on Payments and without a separate history table — the schedule rows, joined with their allocations, already are the installment history.

10. Invoices, Bills and Document Generation

Every completed sale receives a unique invoice. Users can view, print, and download it as a PDF. Installment purchases can additionally generate an installment agreement or schedule, and each installment payment generates a separate receipt.

- Invoice number, date/time, customer, cashier, items, quantities, prices, discounts, tax, total and payment method.
- Serialized-unit details such as serial number or IMEI where applicable.
- Warranty information for eligible electronics.
- Installment deposit, financed amount, interest, installment amount, due dates and remaining balance.

Document metadata is stored in PostgreSQL. Generated PDFs can be stored in S3-compatible object storage such as MinIO during development and a managed object store in production.

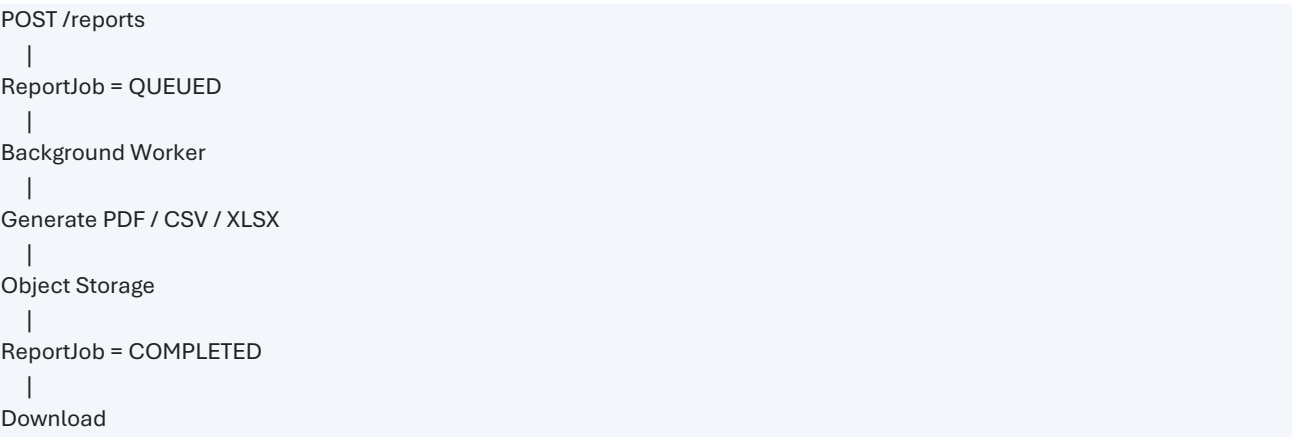
11. Reporting System

The first version uses deterministic reporting rather than an LLM reporting agent. Operational and financial reports are generated from authoritative transactional data and can be filtered and exported.

Report Area	Examples	Exports
Sales	Daily/monthly sales, revenue, product/category sales, cashier performance	PDF / CSV / XLSX
Inventory	Current stock, valuation, movement, low stock, fast/slow movers	PDF / CSV / XLSX
Installments	Collections, upcoming payments, overdue accounts, aging, balances	PDF / CSV / XLSX
Financial	Payment summary, refunds, receivables, daily cash summary	PDF / CSV / XLSX

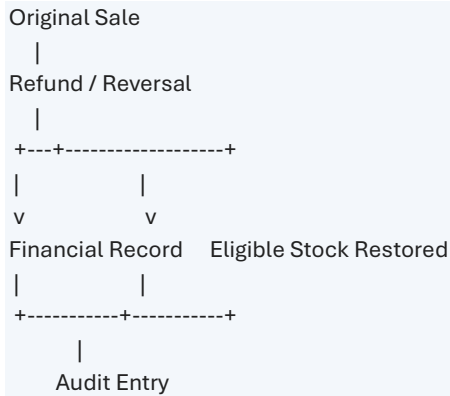
11.1 Background Report Jobs

Large exports should not block an HTTP request. A report job is queued, generated by a worker, saved to object storage, and exposed for download when complete.



12. Refunds and Reversals

Completed sales and payments are never simply deleted. Refunds and reversals create compensating records so the financial history remains traceable.

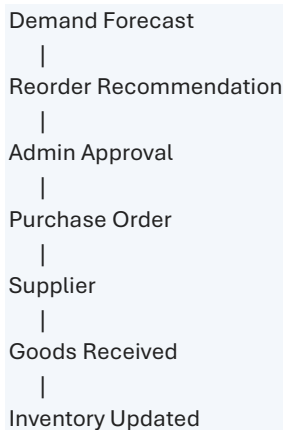


13. Warranty and Returns Management

- Search by invoice number, customer, serial number, or IMEI.
- Display purchase date, warranty duration, expiry date, and current status.
- Register warranty claims and authorized returns.
- Link the claim to the exact serialized unit and original sale.
- Preserve warranty and return history for auditability.

14. Supplier and Procurement Extension

Supplier and purchase-order management is recommended as a later phase. It closes the inventory loop by allowing reorder recommendations to become approved purchase orders and received goods to create inventory movements.



15. Intelligent and ML Components

ML is used for decision support, not for financial transaction processing. Predictions are stored separately and presented to authorized staff as recommendations, scores, or alerts. This entire section is optional: it is attempted only after the core POS, installment, and reporting features are complete and tested. Where attempted, forecasting and risk scoring run as plain Python scripts (no web framework, no live API) that read sales and installment data directly from PostgreSQL and write their output into two dedicated tables, which the backend reads back — there is no synchronous call from the backend into the ML component. Both output tables keep a full history rather than overwriting, one row per forecast or scoring run, so a trend is visible rather than only a latest value.

15.1 Demand Forecasting

Historical sales are used to forecast demand for products or categories over 7-, 14-, and 30-day horizons. Candidate methods include naive forecasting, moving averages, linear models, Random Forest, and XGBoost. Models are compared using MAE, RMSE, and/or MAPE rather than selecting the most complex model by default.

15.2 Stock-Out Risk and Reorder Recommendations

```
Forecast Demand
+
Current Stock
+
Supplier Lead Time
+
Safety Stock
|
v
Stock-Out Risk + Recommended Reorder Quantity
```

The recommendation remains advisory: an administrator approves procurement actions.

15.3 Installment Payment-Risk Scoring

Risk scoring can use previous late payments, average payment delay, missed/partial payments, remaining balance, installment amount, remaining installments, and payment consistency. The output may be LOW, MEDIUM, or HIGH risk. It is used to prioritize reminders and staff follow-up, not to automatically deny financing.

15.4 Anomaly Detection

The system can flag unusual sales, inventory adjustments, refunds, or payment patterns. Initial approaches should use interpretable statistical baselines such as Z-score, IQR, and rolling thresholds before comparing them with methods such as Isolation Forest.

16. Analytics Dashboard

- Today's sales and revenue; monthly revenue; average transaction value.
- Outstanding installment balance and overdue installment count.
- Inventory value, low-stock products, stock-out risk, and top-selling products.
- Forecast trends, reorder recommendations, payment-risk distribution, and anomaly alerts.

Common operational dashboards are built directly in React. Apache Superset can be added for deeper exploratory analytics using PostgreSQL views or materialized views. A lakehouse is unnecessary until the data volume and analytical workload justify it.

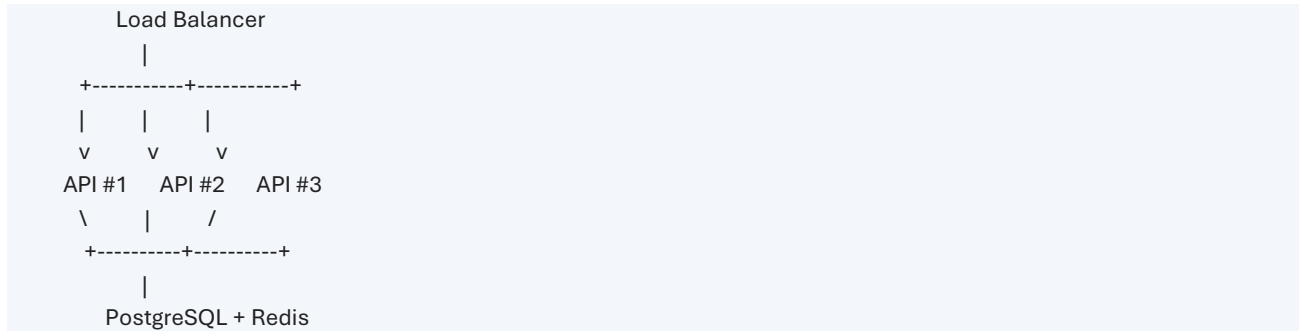
17. Security and Auditability

- Secure authentication and password hashing.
- Role-based access control for Administrator and Worker permissions.
- Request validation, rate limiting, and secure API design.
- Encryption in transit and appropriate encryption at rest.
- Secrets stored outside source control.

- Audit logs for price changes, stock adjustments, payments, refunds, installment changes, and role changes.
- Customer notification consent and opt-out handling.
- Least-privilege access to financial and personal information.

18. Scalability Strategy

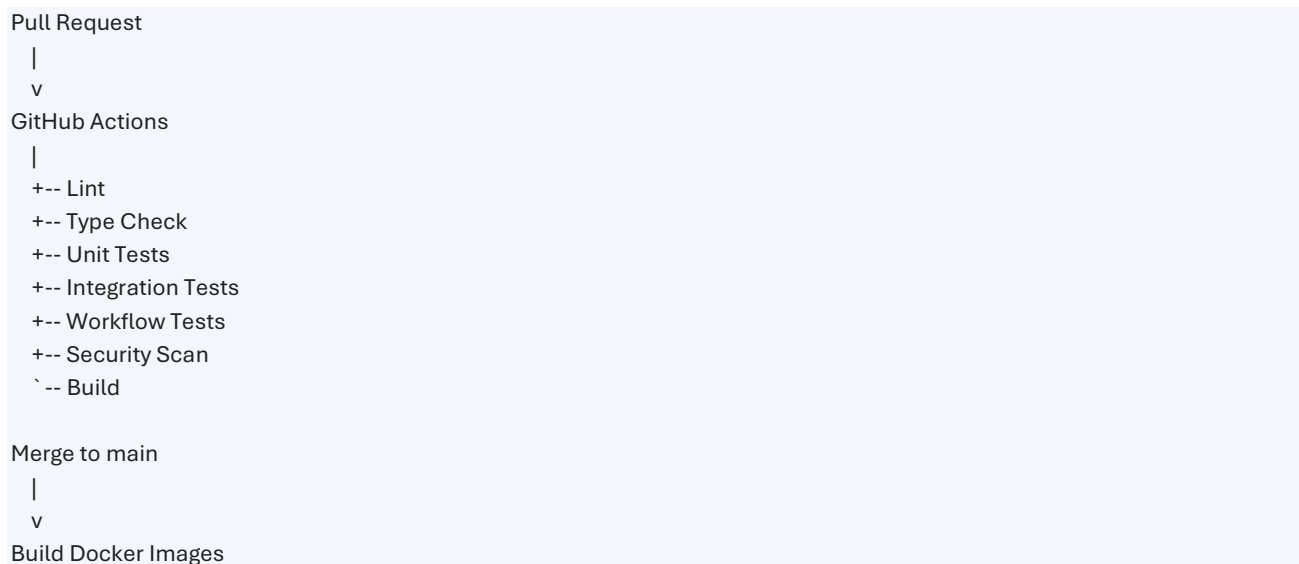
The application is designed to scale horizontally even though Kubernetes is not initially required. API servers remain stateless; shared persistent state lives in PostgreSQL, cache state in Redis, workflow state in Temporal, and documents in object storage.



19. Testing and Reliability Engineering

- Unit tests for installment calculations, payment allocation, discounts, inventory rules, and reorder logic.
- Integration tests for PostgreSQL transactions, Redis, repositories, and Temporal activities.
- API and end-to-end tests for checkout, payment, installment creation, invoice generation, and refunds.
- Concurrency tests for the final available stock unit and duplicate payment submissions.
- Failure tests for notification-provider errors, worker restarts, report-generation crashes, and Redis unavailability.
- ML evaluation against baseline models with reproducible metrics.

20. CI/CD Pipeline



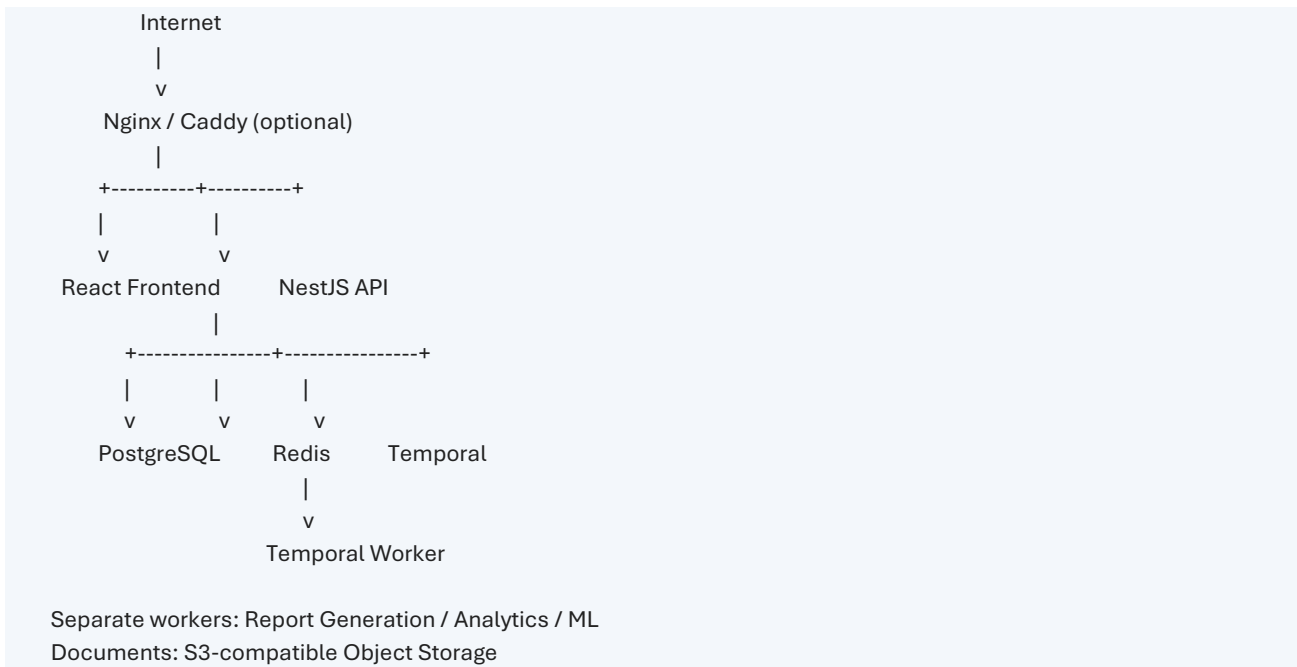
```

|
Container Registry
|
Deploy Staging
|
Smoke Tests
|
Production Deployment

```

Kubernetes can later be used as a separate infrastructure-learning extension, but it is not required to prove that the application is scalable or production-oriented.

21. Deployment Plan



The first deployment can run as Docker containers on a cloud VM, with managed PostgreSQL and object storage used when practical. This provides a realistic production path without introducing unnecessary orchestration overhead. A dedicated reverse proxy (Nginx or Caddy) is only needed for a self-managed VM with a custom domain; if the demo is hosted on a platform that manages HTTPS automatically (e.g. Render, Railway, Fly.io), this component can be skipped entirely.

22. Observability

Area	Metrics / Signals
Application	API latency, requests/sec, 5xx rate, database latency, connection-pool usage, Redis hit ratio
Workflow	Active workflows, failed workflows, retry count, reminder failures, notification failures
Business	Sales/hour, revenue/day, failed payments, overdue installments, refund rate, stock-out rate
ML	Forecast MAE/MAPE, risk-model precision/recall, prediction distributions, data/model drift

OpenTelemetry can standardize telemetry, Prometheus can collect metrics, and Grafana can visualize operational and business health. This full stack is an optional extension, not a core requirement: structured application logging (via the backend's built-in logger) covering key events such as checkout completion, payment failure, and workflow retries is sufficient to demonstrate operational awareness for the core delivery.

23. Recommended Technology Stack

Component	Technology	Purpose
Frontend	React + TypeScript	POS and administration UI
Backend	NestJS + TypeScript	Business logic and REST APIs
Architecture	Modular Monolith	Clear domains without premature microservices
ORM	Prisma	Schema, migrations, and data access
Database	PostgreSQL	Transactional source of truth
Cache	Redis	Caching, rate limiting, short-lived state
Workflow Engine	Temporal	Durable installment workflows
ML / Analytics	Python, Pandas, scikit-learn (optional; plain scripts, no framework)	Forecasting and risk scoring only (optional); reorder suggestions and anomaly flags are a formula and a statistical rule, not ML
Analytics	React dashboards + optional Superset	Operational and exploratory analytics
Documents	Server-side PDF + CSV/XLSX generation	Bills, receipts, statements, reports
Object Storage	MinIO / S3-compatible storage	Generated documents
Containers	Docker + Docker Compose	Development and initial deployment
CI/CD	GitHub Actions	Test, build, scan, deploy
Reverse Proxy	Nginx or Caddy (optional)	TLS termination and routing; skip if hosting on a platform with managed HTTPS
Observability	OpenTelemetry + Prometheus + Grafana (optional)	Metrics, traces, monitoring; core uses structured logging instead

24. Technologies Deliberately Excluded from Phase 1

The following technologies are not required initially because the proposed workload does not justify their operational complexity:

- LLM / vector database - deterministic financial reporting and predictive ML solve the core needs more safely.
- Kafka - no current event throughput requires a distributed streaming platform.
- Microservices - a modular monolith provides better simplicity and transactional coordination.
- Kubernetes / Helm / Argo CD - useful as a later learning or scaling extension, not an application requirement.
- Iceberg / data lakehouse - PostgreSQL reporting views and materialized views are sufficient initially.
- Deep learning - conventional forecasting and tabular ML are more appropriate for the available business data.
- Dedicated reverse proxy (Nginx/Caddy) - only needed for a self-managed VM with a custom domain; unnecessary when deploying behind a platform that manages HTTPS termination automatically.

- Full observability stack (Prometheus + Grafana) - structured application logging is sufficient to demonstrate operational awareness at this project's scale; the full metrics/dashboard stack is deferred to an optional extension.
- Next.js for the frontend - no public or SEO-sensitive pages exist behind login, so server-side rendering has no benefit here; a plain React SPA is simpler.
- FastAPI as the core backend - would mean redoing already-settled work (Prisma, RBAC guards, NestJS modules) for no functional gain; kept only as a possible thin wrapper inside the optional ML component if on-demand forecasting is ever wanted.
- Temporal signals and continue-as-new for the installment workflow - solve problems (instant payment detection, long-running history growth) this system's short, per-installment workflows don't actually have; the simpler polling design already satisfies the requirement.

25. Implementation Roadmap

Phase	Focus	Key Deliverables
Phase 1	Foundation	Repository, React, NestJS, PostgreSQL, Prisma, Docker Compose, migrations.
Phase 2	Security & Users	Authentication, RBAC, validation, audit foundation.
Phase 3	Catalogue & Inventory	Products, categories, stock ledger, serialized units, stock adjustments.
Phase 4	POS & Transactions	Cart, checkout, ACID transactions, locking, idempotency, payments.
Phase 5	Invoices & Documents	PDF bills, printing, object storage, document history.
Phase 6	Installments	Plans, schedules, balances, installment payments and receipts.
Phase 7	Temporal Workflows	Timers, reminders, retries, overdue processing, failure recovery.
Phase 8	Notifications	SMS/WhatsApp abstraction, reminders, receipts, consent handling.
Phase 9	Refunds & Warranty	Reversals, returns, warranty lookup and claims.
Phase 10	Reporting	Sales, inventory, installment and financial reports; PDF/CSV/XLSX.
Phase 11	Analytics	KPIs, dashboards, PostgreSQL views/materialized views, optional Superset.
Phase 12	Inventory Intelligence	Sales velocity, stock coverage, stock-out risk, reorder logic.
Phase 13	Machine Learning	Demand forecasting, risk scoring, anomaly detection and evaluation.
Phase 14	Testing & Reliability	Concurrency, failure, workflow, integration and E2E testing.
Phase 15	CI/CD & Deployment	GitHub Actions, image builds, staging, smoke tests, Docker production deployment.
Phase 16	Observability (Optional)	OpenTelemetry, Prometheus, Grafana, business and ML metrics. Core delivery uses structured logging only; this phase is attempted only if time remains.
Phase 17	Optional Expansion	Suppliers/procurement, Kubernetes practice deployment, advanced analytics.

26. Expected Deliverables

- Functional electronics POS with full-payment and installment sales.
- Transaction-safe payment processing and inventory updates.
- Serialized inventory, warranty tracking, returns, refunds, and reversals.
- Temporal-based durable installment workflows with reminders and overdue escalation.

- PDF invoices, installment receipts, statements, and downloadable PDF/CSV/XLSX reports.
- Administrative dashboards and operational analytics.
- Demand forecasting, stock-out prediction, reorder recommendations, installment-risk scoring, and anomaly detection.
- RBAC, audit logging, idempotency, concurrency controls, and secure notification consent handling.
- Automated unit, integration, workflow, concurrency, failure, and E2E tests.
- Dockerized development/deployment, GitHub Actions CI/CD, and structured application logging, with a full production observability stack as an optional extension.

27. Expected Technical Value

The project goes beyond a conventional CRUD POS by combining database correctness, financial transaction design, workflow orchestration, domain-specific electronics inventory, document generation, applied machine learning, DevOps, and observability in one coherent system.



28. Final Design Principle

Use the simplest technology that reliably solves each problem, while designing clear boundaries that allow the system to scale when real requirements justify additional complexity.

In SmartRetail, PostgreSQL owns transactional truth; NestJS owns business rules; Temporal owns long-running workflow state; Redis supports cache and short-lived state; Python provides predictive decision support; object storage holds generated documents; Docker and GitHub Actions support reproducible delivery; and OpenTelemetry, Prometheus, and Grafana provide operational visibility.